

## Laboratory Quiz / Viva Answers

*Short answers to the quiz questions of all 12 experiments — for hand-written submission*

| Particulars            | Details                                         |
|------------------------|-------------------------------------------------|
| Name of the student    |                                                 |
| Enrolment No. · Branch | Robotics and Automation                         |
| Subject                | Microcontroller and Microprocessor (BE05041011) |
| Experiments covered    | All 12 (8085 · 8051 · AVR)                      |
| Date                   |                                                 |

### Experiment 1 — Introduction to Microprocessor (8085)

#### 1. What are the various registers available in 8085?

**Ans.** General-purpose registers **B, C, D, E, H, L** (8-bit, paired as BC, DE, HL); **Accumulator (A)**; **Flag register**; **Program Counter (PC, 16-bit)**; **Stack Pointer (SP, 16-bit)**; temporary registers W and Z; and the Instruction Register.

#### 2. What are the various flags used in 8085?

**Ans.** Five flags — **Sign (S)**, **Zero (Z)**, **Auxiliary Carry (AC)**, **Parity (P)** and **Carry (CY)**.

#### 3. What is stack pointer?

**Ans.** A 16-bit register that holds the address of the **top of the stack** in memory.

#### 4. What is program counter?

**Ans.** A 16-bit register that holds the address of the **next instruction** to be executed.

#### 5. Define interrupt.

**Ans.** A signal that stops the current program and transfers control to a service routine (ISR); after servicing, control returns to the main program.

#### 6. What are the hardware & software interrupts?

**Ans.** **Hardware interrupts** are caused by external signals on pins — TRAP, RST 7.5, RST 6.5, RST 5.5, INTR. **Software interrupts** are caused by instructions in the program — RST 0 to RST 7.

#### 7. What is the clock frequency for 8085?

**Ans.** Internal operating frequency is **3 MHz** (maximum ~3.072 MHz); the crystal is 6 MHz and is divided by 2 internally.

#### 8. What is ALE? Explain the function of ALE in 8085.

**Ans.** **ALE = Address Latch Enable.** It demultiplexes the lower-order address/data bus AD0-AD7. When ALE = 1, the bus carries the lower address byte, which is latched into an external latch (74LS373); the same lines then carry data.

## Experiment 2 — Data Transfer Instructions

---

### 1. Write the names of addressing modes.

**Ans.** The 8085 has five — **Immediate, Direct, Register, Register Indirect** and **Implied (Implicit)**.

### 2. List any three data transfer instructions with immediate addressing mode.

**Ans.** MVI A, 05H · LXI H, 2000H · MVI B, 0FH (data given directly in the instruction).

### 3. Explain the operation of PUSH & POP instruction with a suitable program.

**Ans.** **PUSH rp** stores a register pair on the stack (SP decreases by 2); **POP rp** retrieves a pair from the stack (SP increases by 2). e.g. LXI SP, 27FFH → PUSH B (saves BC) → POP D (loads DE from stack).

## Experiment 3 — Arithmetic and Logical Instructions

---

### 1. What is the purpose of the ADD instruction in 8085?

**Ans.** It adds the contents of a register/memory to the accumulator and stores the result in the accumulator ( $A = A + R$ ).

### 2. Which 8085 instruction is used for logical AND operations?

**Ans.** **ANA** (ANA R / ANA M) and **ANI** (AND with immediate data).

### 3. Which instruction decrements a register or memory location?

**Ans.** **DCR** (e.g. DCR B, DCR M) for 8-bit; **DCX** for a register pair.

### 4. What is the role of the CMP instruction?

**Ans.** It compares the accumulator with a register/memory by subtraction; the result is **not stored**, only the flags (Z, CY) are affected.

### 5. Which instruction is used to clear a specific bit in a register/memory?

**Ans.** Use **ANI/ANA with a mask** having 0 at that bit position and 1 elsewhere (e.g. ANI 0FEH clears bit 0).

### 6. What is the purpose of the XRA instruction?

**Ans.** Bitwise **Exclusive-OR** of the accumulator with a register/memory. XRA A clears the accumulator ( $A = 00H$ ).

### 7. Which instruction performs addition with carry?

**Ans.** **ADC** (ADC R / ADC M) and **ACI** (add immediate with carry).

## Experiment 4 — Branching and Looping Instructions

---

### 1. What is the difference between conditional & unconditional branching instructions?

**Ans.** **Unconditional** branches always transfer control (JMP, CALL, RET). **Conditional** branches transfer control only if a flag condition is satisfied (JZ, JNZ, JC, JNC, JP, JM, etc.).

### 2. What is the difference in operation of JMP & CALL instruction?

**Ans.** **JMP** transfers control to a new address without saving a return address. **CALL** transfers control to a subroutine and **saves the return address on the stack**, so RET can return to the main program.

## Experiment 5 — Data Transfer Programming in 8051

---

### 1. List the available software for the microcontroller practical.

**Ans.** Keil  $\mu$ Vision, SimulIDE, Proteus, MIDE-51, EdSim51, and the AVR/8051 assembler toolchains.

### 2. Describe the dissimilarities between internal and external memory in the 8051.

**Ans.** **Internal memory** is on-chip (256 bytes RAM + 4 KB ROM), fast but limited.

**External memory** is off-chip (up to 64 KB RAM + 64 KB ROM), larger but slower, accessed through ports P0/P2 using the **MOVX / MOVC** instructions.

## Experiment 6 — Arithmetic Instruction Programming in 8051

---

### 1. Describe the purpose of studying data transfer and arithmetic operations on external memory.

**Ans.** To learn how to store and retrieve data in external RAM (via MOVX) and perform arithmetic on it — needed when the on-chip memory is insufficient for the application.

### 2. Define external memory.

**Ans.** Memory located **outside the microcontroller chip** (external RAM/ROM, up to 64 KB each), accessed through the address and data buses using the MOVX and MOVC instructions.

## Experiment 7 — Boolean & Logical Instructions Programming (8051)

---

### 1. Explain the purpose of the ANL instruction in 8051 programming.

**Ans.** It performs a bitwise **logical AND** between the destination (A or a direct byte) and the source, storing the result in the destination. It is mainly used for **masking / clearing bits**.

### 2. Illustrate how to clear a bit in a byte using the 8051.

**Ans.** AND the byte with a mask that has 0 at the bit to clear and 1 elsewhere — e.g. ANL A, #0FEH clears bit 0. For a bit-addressable location use CLR directly, e.g. CLR P1.0.

## Experiment 8 — Blinking LED using 8051

---

### 1. What is the 8051 microcontroller?

**Ans.** An **8-bit single-chip microcontroller** with 4 KB ROM, 128 bytes RAM, four 8-bit I/O ports, two timers, a serial port and interrupts.

### 2. What is the function of Port 1?

**Ans.** Port 1 (P1) is an 8-bit **bidirectional general-purpose I/O port**.

### 3. Which pin of Port 1 is used for the LED?

**Ans.** **P1.0** (the first pin of Port 1).

### 4. Why is a resistor connected in series with the LED?

**Ans.** To **limit the current** through the LED and protect it from damage (typically 220–330  $\Omega$ ).

### 5. What is the purpose of the crystal oscillator?

**Ans.** It provides the **clock signal** that sets the operating and timing frequency of the microcontroller.

**6. Why is 11.0592 MHz commonly used with the 8051?**

**Ans.** Because it divides evenly to give **exact standard baud rates** for serial (UART) communication.

**7. What is the purpose of ORG 0000H?**

**Ans.** An assembler directive that sets the program's **starting address at 0000H**, the reset vector of the 8051.

**8. How to create a hex file?**

**Ans.** Write and assemble/compile the source in an IDE (e.g. Keil) and **build the project** — the assembler generates the .hex file.

**9. How many ports does the 8051 have?**

**Ans.** Four 8-bit I/O ports — **P0, P1, P2, P3** (32 I/O lines in total).

---

Experiment 9 — AVR Arithmetic & Logical Operations (Viva)

---

**1. What is the purpose of the LDI instruction?**

**Ans. Load Immediate** — loads an 8-bit constant into a register R16–R31, e.g. LDI R16, 0x05.

**2. Where is the result of the MUL instruction stored?**

**Ans.** In the register pair **R1:R0** — high byte in R1, low byte in R0.

**3. Which AVR instruction performs XOR?**

**Ans. EOR** (Exclusive OR), e.g. EOR R16, R17.

**4. What does the COM instruction do?**

**Ans.** It takes the **one's complement** of a register — inverts all bits ( $R = 0xFF - R$ ).

**5. What is a HEX file?**

**Ans.** A file in **Intel HEX format** holding the machine code in hexadecimal, which is loaded into the microcontroller's program (flash) memory.

---

Experiment 10 — Read Push-Button Switch & Display on LED (8051)

---

**1. Which port is used to read the push button?**

**Ans. Port 1** — the switch is connected to pin **P1.0**.

**2. Which pin is used to control the LED?**

**Ans. P2.0.**

**3. Why is FFH written to Port 1?**

**Ans.** Writing MOV P1, #0FFH sets all P1 pins high so the port acts as an **input**, letting P1.0 read the switch.

**4. What happens to P1.0 when the switch is pressed?**

**Ans.** It is connected to ground, so **P1.0 reads logic 0 (LOW)**; the LED at P2.0 then turns ON.

**5. What is the function of the JB instruction?**

**Ans. Jump if Bit = 1** — it checks a bit and jumps if that bit is set (e.g. JB P1.0, NOT\_PRESSED).

## Experiment 11 — Interfacing Buzzer with AVR Board

---

### 1. What is the function of a buzzer in an embedded system?

**Ans.** It is an **output device** that produces an audible sound/tone for alarms, alerts or status indication.

### 2. Why is PB0 configured as an output in this experiment?

**Ans.** Because the buzzer is driven by PB0; it must be an output to **send HIGH/LOW signals** to turn the buzzer ON and OFF.

### 3. What is the purpose of the DDRB register?

**Ans. Data Direction Register for Port B** — it sets each pin as input (0) or output (1).  
`DDRB |= (1<<PB0)` makes PB0 an output.

### 4. What is the difference between an active buzzer and a passive buzzer?

**Ans.** An **active buzzer** has a built-in oscillator and sounds on a steady DC voltage. A **passive buzzer** has no oscillator and needs an external AC/PWM signal to produce sound.

### 5. What is the function of PORTB in this program?

**Ans.** It sets the output level of Port B pins — `PORTB |= (1<<PB0)` drives PB0 HIGH (buzzer ON) and `PORTB &= ~(1<<PB0)` drives it LOW (buzzer OFF).

## Experiment 12 — Interfacing 7-Segment Display with AVR Board

---

### 1. What is a 7-segment display?

**Ans.** An LED display made of **seven segments (a-g)** plus a decimal point, which are lit in combinations to show digits 0-9 and some characters.

### 2. What is the difference between common-cathode and common-anode displays?

**Ans.** In a **common-cathode** display all cathodes join to ground and a segment glows when its anode is driven HIGH. In a **common-anode** display all anodes join to +V and a segment glows when driven LOW.

### 3. Why are 330 $\Omega$ resistors used with the LED segments?

**Ans.** As **current-limiting resistors** in series with each segment to protect the LEDs from excess current.

### 4. Which ATmega32 port is used in this experiment?

**Ans. PORTD** — PD0-PD7 are mapped to segments a-g and dp.

### 5. What does `DDRD = 0xFF` do?

**Ans.** It configures **all eight pins of Port D as outputs**.

### 6. Why is `F_CPU` defined as 8 MHz?

**Ans.** It tells the compiler the CPU clock is **8 MHz** so that `_delay_ms()` produces accurate delays.

### 7. What hexadecimal code is used to display digit 8?

**Ans. 0x7F** (all seven segments a-g ON).